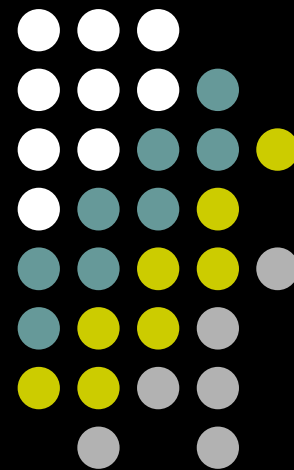


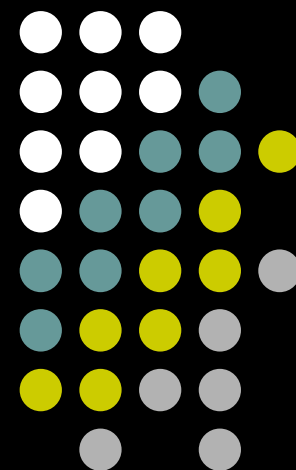
第5章 循环结构程序设计

Part B

- 5.7 改变循环执行的状态
- 5.8 循环程序举例



5.7 改变循环执行的状态





break语句

- break语句的一般形式：
`break;`
- break语句的功能：
 1. 在switch语句中使用，使流程跳出switch结构，继续执行switch语句后面的语句。
 2. 在for、while或do-while语句使用，使流程从循环体内跳出，即**提前结束循环**，接着执行后面的语句。

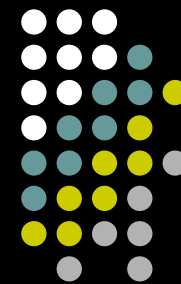
【例5.4】在全系1000学生中，征集慈善募捐，当总数达到10万元时就结束，统计此时捐款的人数，以及平均每人捐款的数目。



【编程思路】

- 循环次数不确定，但最多循环1000次
 - 在循环体中累计捐款总数
 - 用if语句检查是否达到10万元
 - 如果达到就不再继续执行循环，终止累加
- 计算人均捐款数

【例5.4】在全系1000学生中，征集慈善募捐，当总数达到10万元时就结束，统计此时捐款的人数，以及平均每人捐款的数目。



【编程思路】

- 变量amount，用来存放捐款数
- 变量total，用来存放累加后的总捐款数
- 变量aver，用来存放人均捐款数
- 定义符号常量SUM代表100000

```
#include <stdio.h>
```

```
#define SUM 10000
```

```
int main()
```

```
{
```

```
    float amount, aver;
```

```
    int i;
```

```
    for (i=1, total=0;
```

```
    {
```

```
        printf("please
```

```
        scanf("%f", &
```

```
        total= total+amount;
```

```
        if (total >= SUM)
```

```
            break;
```

```
    }
```

```
    aver = total / i;
```

```
    printf("num = %d\naver = %10.2f\n", i, aver);
```

```
    return 0;
```

```
}
```

指

```
please enter amount:12000
```

```
please enter amount:24600
```

```
please enter amount:3200
```

```
please enter amount:5643
```

```
please enter amount:21900
```

```
please enter amount:12345
```

```
please enter amount:23000
```

```
num=7
```

```
aver= 14669.71
```

达到10万，提前结束循环

只能用于循环语句和switch
语句之中，而不能单独使用

实际捐款人数



continue语句

- continue语句的一般形式:

`continue;`

- continue语句的功能:

在for、while或do-while语句使用，用于**结束本次循环体的执行**，即跳过循环体中该语句之后尚未执行的语句:

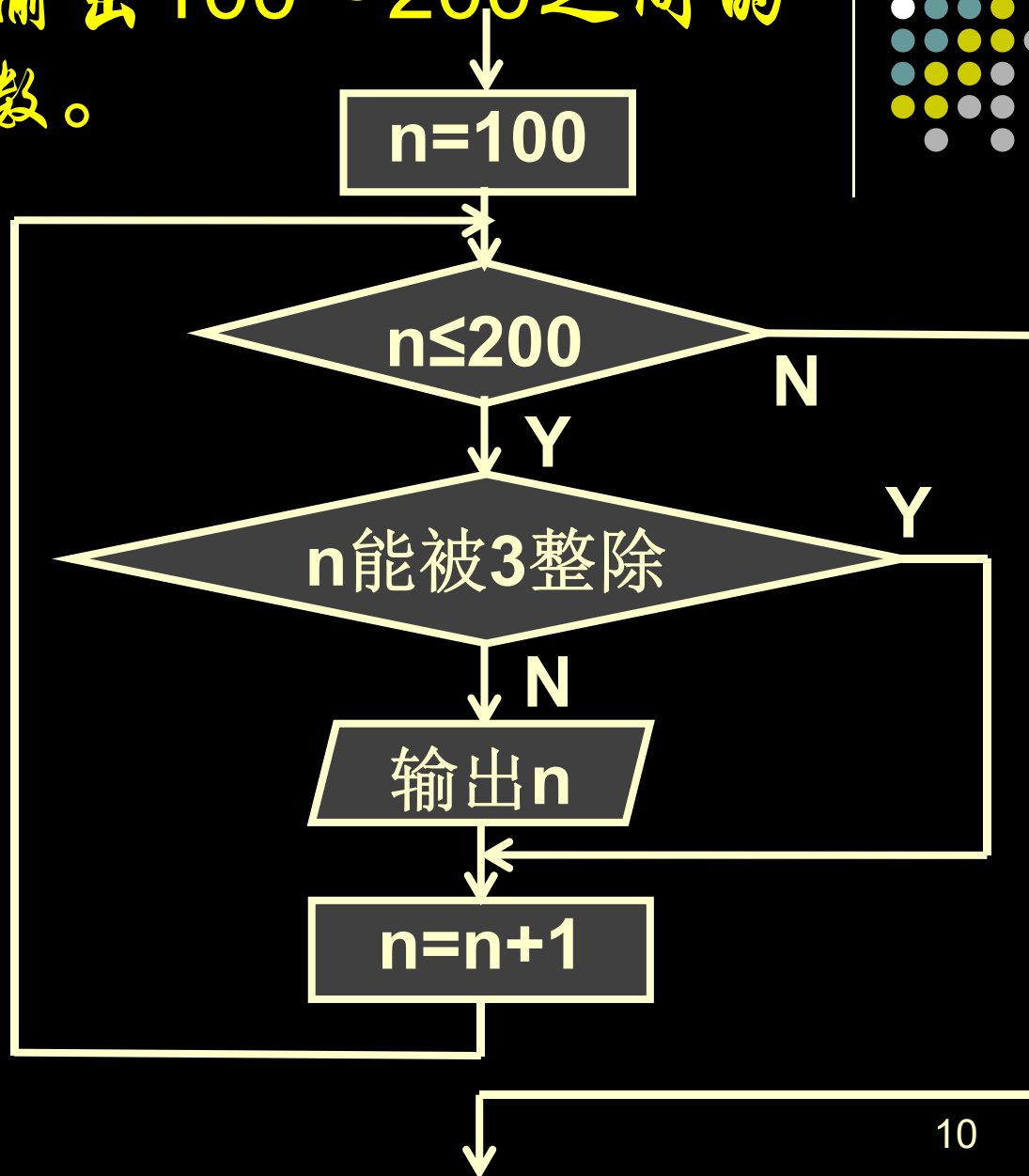
- while和do-while语句中，**直接进入下一次是否执行循环体的判断**（即对循环判断表达式求值）；
- for语句中，则先对表达式3求值，再进入下一次是否执行循环体的判断（即对表达式2求值）

【例5.5】 要求输出100~200之间的不能被3整除的数。



【编程思路】

- 对100到200之间的每一个整数进行检查
- 如果不能被3整除，输出，否则不输出
- 无论是否输出此数，都要接着检查下一个数(直到200为止)。

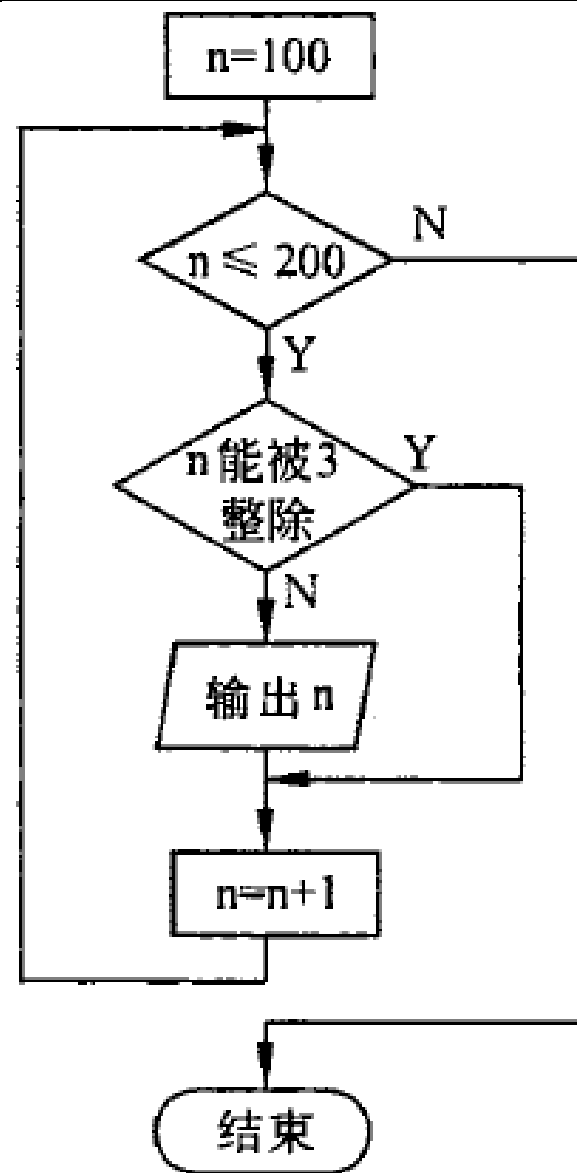



```

#include <stdio.h>

int main()
{
    int n;
    for (n=100; n<=200; n++)
    {
        if (n%3==0)
        {
            continue;
            printf("%d", n);
        }
        if (n%3!=0)
            printf(" %d ", n);
    }
    printf("\n");
    return 0;
}

```



```

100 101 103 104 106 107 109 110 112 113 115 116 118 119 121 122
124 125 127 128 130 131 133 134 136 137 139 140 142 143 145 146
148 149 151 152 154 155 157 158 160 161 163 164 166 167 169 170
172 173 175 176 178 179 181 182 184 185 187 188 190 191 193 194
196 197 199 200

```

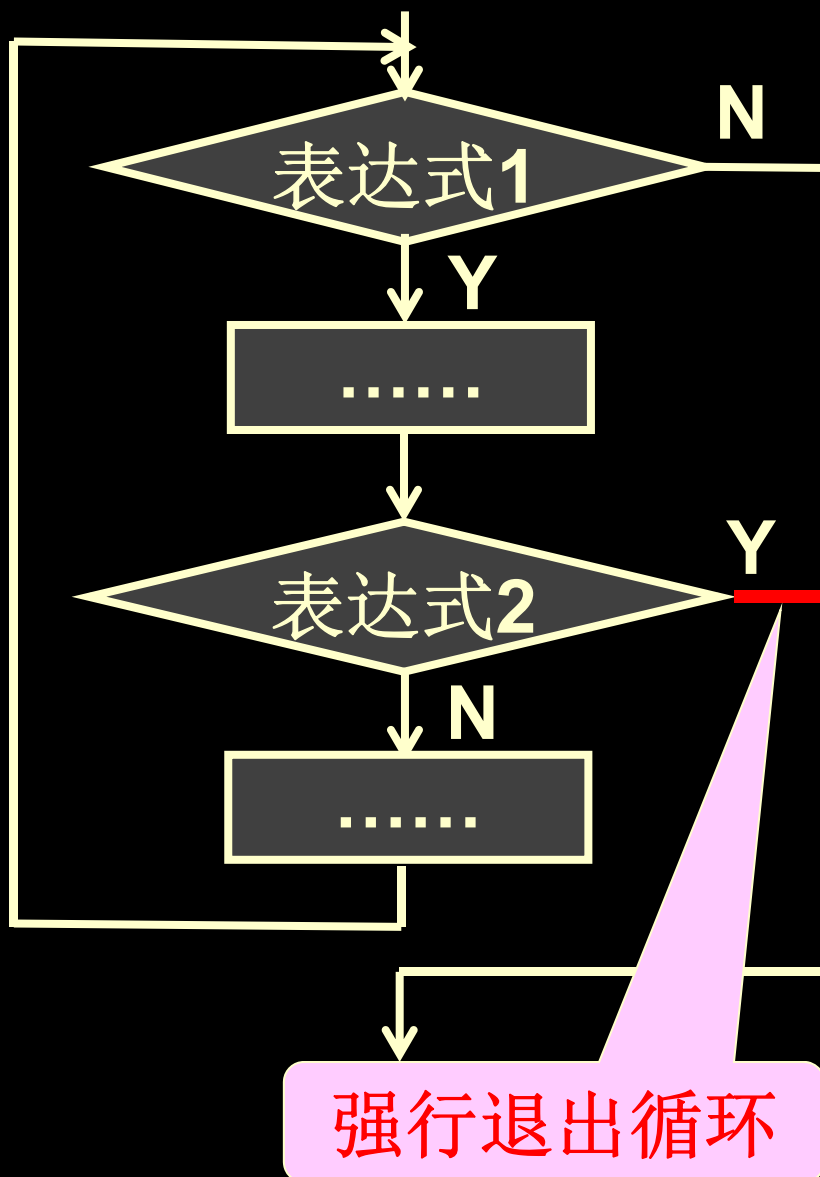
5.7 break语句和continue语句（3）



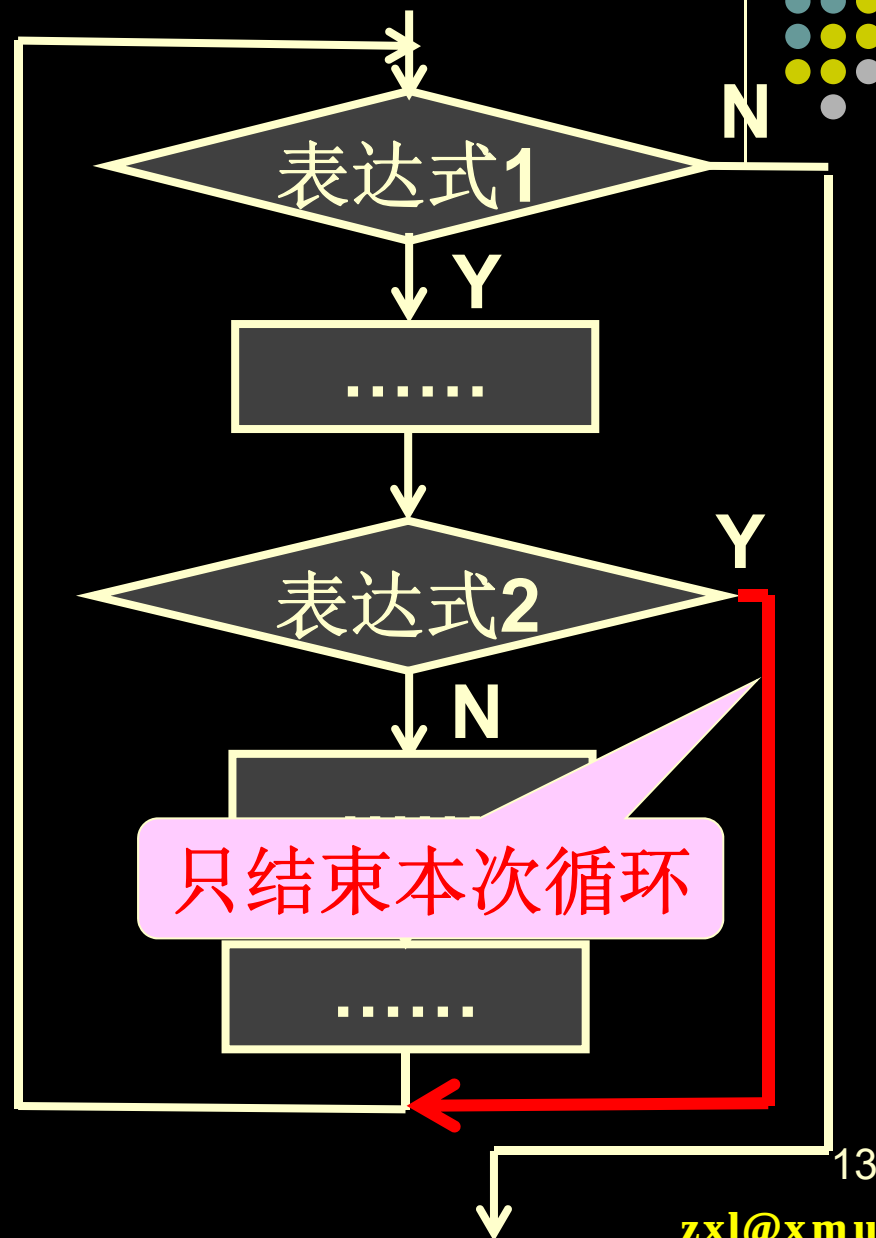
- continue 语句与break语句的区别：
 - continue 语句只是结束本次循环，而不是终止整个循环语句的执行。
 - break语句结束的是整个循环语句的执行。

【例】 对比P129的图5—13与图5—14

break语句



continue语句



比较下列程序的输出结果



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for (i=0; i<5; i++)
```

```
    {
```

```
        for (j=0; j<5 ; j++)
```

```
        {
```

```
            printf("*");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
*****
```

```
*****
```

```
*****
```

```
*****
```

```
*****
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i,j;
```

```
    for (i=0; i<5; i++)
```

```
    {
```

```
        for (j=0; j<=i; j++)
```

```
        {
```

```
            printf("*");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
**
```

```
***
```

```
****
```

```
*****
```

```
*****
```

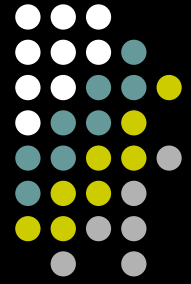
比较下列程序的输出结果 (2)

```
#include <stdio.h>
int main()
{
    int i,j;
    for (i=0; i<5; i++)
    {
        for (j=0; j<5; j++)
        {
            if (j==i+1)
                break;
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

```
**
***
****
*****
```

```
#include <stdio.h>
int main()
{
    int i,j;
    for (i=0; i<5; i++)
    {
        for (j=0; j<5; j++)
        {
            if (j==i+1)
                continue;
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

```
*****
*****
*****
*****
*****
```



【例5.6】 输出以下4*5的矩阵。

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20

【解题思路】

- 可以用循环的嵌套来处理此问题
- 用外循环来输出一行数据
- 用内循环来输出一列数据
- 按矩阵的格式(每行5个数据)输出



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, j, n=0;
```

输出4行

```
    for (i=1; i<=4; i++)
```

累计输出数据的个数

```
        for (j=1; j<=5; j++, n++)
```

```
        {
```

每行中输出5个数据

```
            if (n % 5 == 0)
```

```
                printf ("\n");
```

```
                printf ("%d\t", i*j);
```

```
        }
```

每输出5个数据换行一次

```
    printf ("\n");
```

双重循环

```
    return 0;
```

```
}
```

#i	1	2	3	4	5
int	1	2	3	4	5
{	3	4	6	9	10
	4	8	12		

如何修改程序，不输出第一行的空行？

```

for (i=1; i<=4; i++)
    for (j=1; j<=5; j++, n++)
    {
        if (n%5==0 && n>0)
            if (n % 5 == 0)
                printf ("\n");
                printf ("%d\t", i*j);
    }
    printf ("\n");
return 0;
}

```

i=1时 **i=2时**

j由1增加到5

i*j的值依次是1,2,3,4,5

j也由1增加到5

i*j的值依次是2,4,6,8,10

依此类推.....



1	2	3	4	5
2	4	6	8	10
4	8	12	16	20

```
for (i=1; i<=4; i++)  
    for (j=1; j<=5; j++, n++)
```

遇到第3行第1列，
终止内循环

```
        if (n % 5 == 0)  
            printf ("\n");  
        if (i==3 && j==1) break;  
        printf ("%d\t", i*j);
```

```
    }  
    printf ("\n");  
    return 0;
```

```
}
```

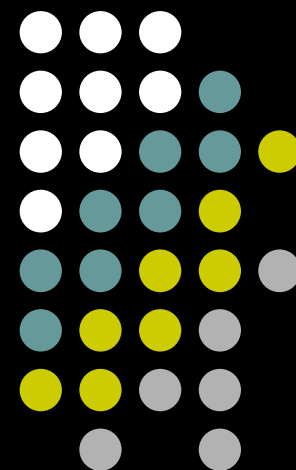


1	2	3	4	5
2	4	6	8	10
6	9	12	15	
4	8	12	16	20

```
for (i=1; i<=4; i++)
    for (j=1; j<=5; j++, n++)
    {
        if (n % 5 == 0)
            printf ("\n");
        if (i==3 && j==1) continue;
        printf ("%d\t", i*j);
    }
    printf ("\n");
return 0;
}
```

原来第3行第1
个数据3没有
输出

5.6 几种循环的比较





几种循环的比较

- 3种循环在流程控制能力上是相同的，一般情况下可以互相代替。
- 在循环实现细节上的差异：
 1. **使循环趋向于结束的操作**：while和do-while循环应在循环体内包含这类语句，而for循环可以在表达式3中实现，甚至将这个循环体都放到表达式3中；
 2. **循环变量初始化操作**：while和do-while循环必须在语句之前完成，而for循环可以在表达式1中实现；
 3. **break语句和continue语句的使用**：while、do-while和for循环中均可使用。

几种循环的比较（2）



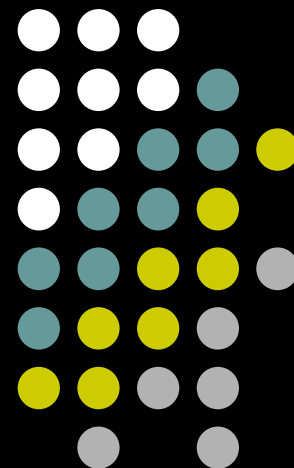
- 3种循环中，for循环和while循环的使用率最高；for语句的功能最强，凡用while循环能完成的，用for循环都能实现。
- 用while循环还是for循环，主要根据个人的爱好，但一般在不用初始化和重新修改变量时用while循环比较恰当，如：

```
while ( (c = getchar( )) == ' '
        || c == '\n' || c == '\t' );
```

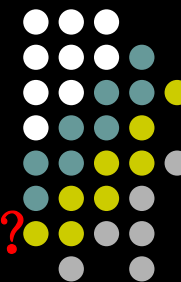
反之，使用for更好些，如：

```
for (i = 0; i < n; i++) {…};
```

5.8 循环程序举例

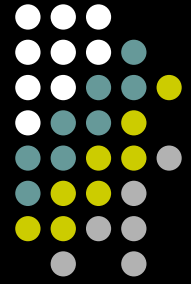


如何写好循环?



1. 需要发现计算过程中可能需要（应该使用）循环。How?
 - 在分析问题时，注意识别出计算过程中重复执行的类似动作（重复性计算）。——重要线索
 - 一批可按统一规律计算的数据；
 - 需要对一系列类似数据做同样的处理；
 - 需要反复从一个结果算出另一个结果，等等。
 - 如果重复次数较多，或是重复的次数无法确定，就应考虑用循环结构描述。
2. 需要考虑和解决许多具体问题：
 - 循环中涉及哪些变量？
 - 循环开始前应该赋给它们什么初值？
 - 循环体中应该如何修改它们？
 - 在什么情况下应该继续（或终止）循环？
 - 循环终止后如何得到所需要的结果？
 -

循环中的几种变量



- 循环控制变量（简称循环变量）
 - 在循环开始之前设置初始值，每次循环执行时递增/减一个固定值，直到其值达到/超过某个界限时循环结束。
 - 控制着循环的进行和结束，使之成为具有有限执行次数的循环。
- 累积变量
 - 在每次循环执行中被更新，其更新常常可以用“+=”或“*=”一类运算符来描述。
 - 循环开始之前的初值通常置为相应运算的0元。
 - 循环结束时所保存的值（终止值）即为循环计算的最终结果。
- 递推变量（循环变量和累积变量都是其中的特例）
 - 在循环中互相协调工作的多个变量，它们亦步亦趋，每次循环均通过其中一个或几个算出另一个或几个新值，然后按某种顺序更新各个变量。

控制循环的进行和终止的典型方法



- 通过计数器控制循环
- 用结束标志控制循环
 - 特定的条件、标志值等等
- 根据输入控制循环
 - 输入的数据、输入函数的返回值等等

例5.7用 $\frac{\pi}{4} \approx 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$ 公式求 π 的近似值，直到发现某一项的绝对值小于 10^{-6} 为止(该项不累计加)。



- 解题思路：
 - 求 π 近似值的方法很多，本题是一种
 - 其他方法：

$$\pi \approx \frac{22}{7}$$

$$\frac{\pi^2}{6} \approx \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \dots + \frac{1}{n^2}$$

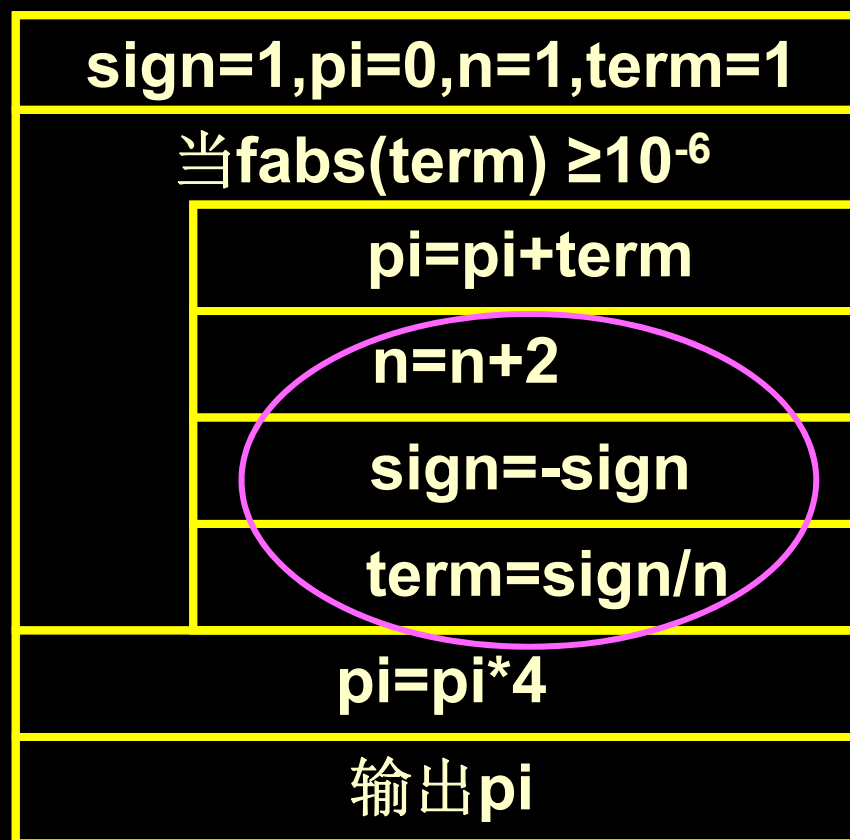
$$\frac{\pi}{2} = \frac{2 \times 2}{1 \times 3} \times \frac{4 \times 4}{3 \times 5} \times \frac{6 \times 6}{5 \times 7} \times \dots \times \frac{(n-1)^2}{n \times (n+2)}$$

例5.7用 $\frac{\pi}{4} \approx 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$ 公式求 π 的近似值，直到发现某一项的绝对值小于 10^{-6} 为止(该项不累加)。



- 解题思路：
 - 每项的分子都是1
 - 后一项的分母是前一项的分母加2
 - 第1项的符号为正，从第2项起，每一项的符号与前一项的符号相反

$$\frac{1}{n} \rightarrow -\frac{1}{n+2}$$



```

#include <stdio.h>
#include <math.h>
int main()
{
    int sign=1;
    double pi=0,n=1,term=1;
    while(fabs(term) >= 1e-6)
    {
        pi=pi+term;
        n=n+2;
        sign=-sign;
        term=sign/n;
    }
    pi=pi*4;
    printf("pi= %10.8f\n",pi);
    return 0;
}

```

改为1e-8

双精度数求绝对值的函数

(整数用a

sign=1,pi=0,n=1,term=1

当fabs(term) ≥ 10⁻⁶

pi=pi+term

n=n+2

sign=-sign

term=sign/n

pi=pi*4

输出pi

pi=3.14159265

循环50万次

只保证前5位小数是准确的

pi=3.14159263

循环5000万次

32

zxl@xmu

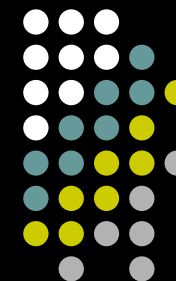
【例5.8】求费波那西(Fibonacci)数列的前40个数。这个数列有如下特点：第1、2两个数为1、1。从第3个数开始，该数是其前面两个数之和。即：

$$\begin{cases} F_1 = 1 & (n = 1) \\ F_2 = 1 & (n = 2) \\ F_n = F_{n-1} + F_{n-2} & (n \geq 3) \end{cases}$$

这是一个有趣的古典数学问题：

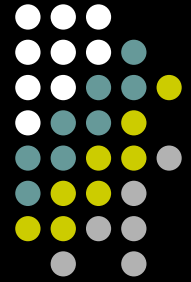
- 有一对兔子，从出生后第3个月起每个月都生一对兔子。
- 小兔子长到第3个月后每个月又生一对兔子。
- 假设所有兔子都不死，问每个月的兔子总数为多少？





第几个月	小兔子对数	中兔子对数	老兔子对数	兔子总数
1	1	0	0	1
2	0	1	0	1
3	1	0	1	2
4	1	1	1	3
5	2	1	2	5
6	3	2	3	8
7	5	3	5	13
⋮	⋮	⋮	⋮	⋮

$$\begin{cases} F_1 = 1 & (n = 1) \\ F_2 = 1 & (n = 2) \\ F_n = F_{n-1} + F_{n-2} & (n \geq 3) \end{cases}$$



f1=1,f2=1

输出f1,f2

For i=1 to 38

f3=f1+f2

输出f3

f1=f2

f2=f3



```
#include <stdio.h>
```

```
int main()
```

```
{ int f1=1,f2=1,f3; int i;
```

```
    printf(" %12d\n%12d\n",f1,f2);
```

```
    for(i=1; i<=38; i++)
```

```
    {
```

```
        f3=f1+f2;
```

```
        printf(" %12d\n",f3);
```

```
        f1=f2;
```

```
        f2=f3;
```

```
    }
```

```
    return 0;
```

```
}
```

1

1

2

3

5

8

13

21

34

55

89

代码可改进


```
#include
int
{
```

$f_1 = 1, f_2 = 1$

for $i = 1$ to 20

输出 f_1, f_2

$f_1 = f_1 + f_2$

$f_2 = f_2 + f_1$

$t = i;$

$f_1 = f_1 + f_2$

```
printf("%12d\n%12d\n", f1, f2);
```

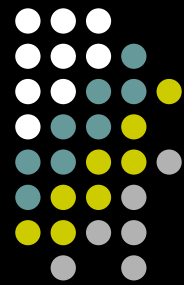
```
    f1 = f1 + f2;
```

```
    f2 = f2 + f1;
```

```
}
```

```
return 0;
```

```
}
```



1
1
2
3
5
8
13
21
34
55
89

...

zxl@xmu

```
#include <stdio.h>
int main()
{ int f1=1,f2=1; int i;
  for(i=1; i<=20; i++)
  {
```

```
    printf(" %12d %12d ",f1,f2);
```

```
    if(i%2==0)
```

```
        printf("\n");
```

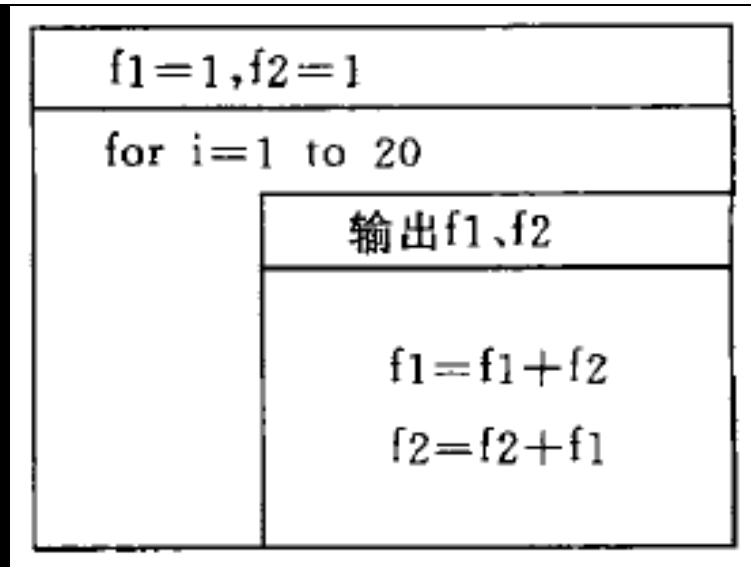
```
    f1=f1+f2;
```

```
    f2=f2+f1;
```

```
}
```

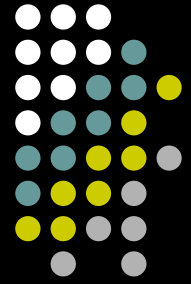
```
return 0;
```

```
}
```



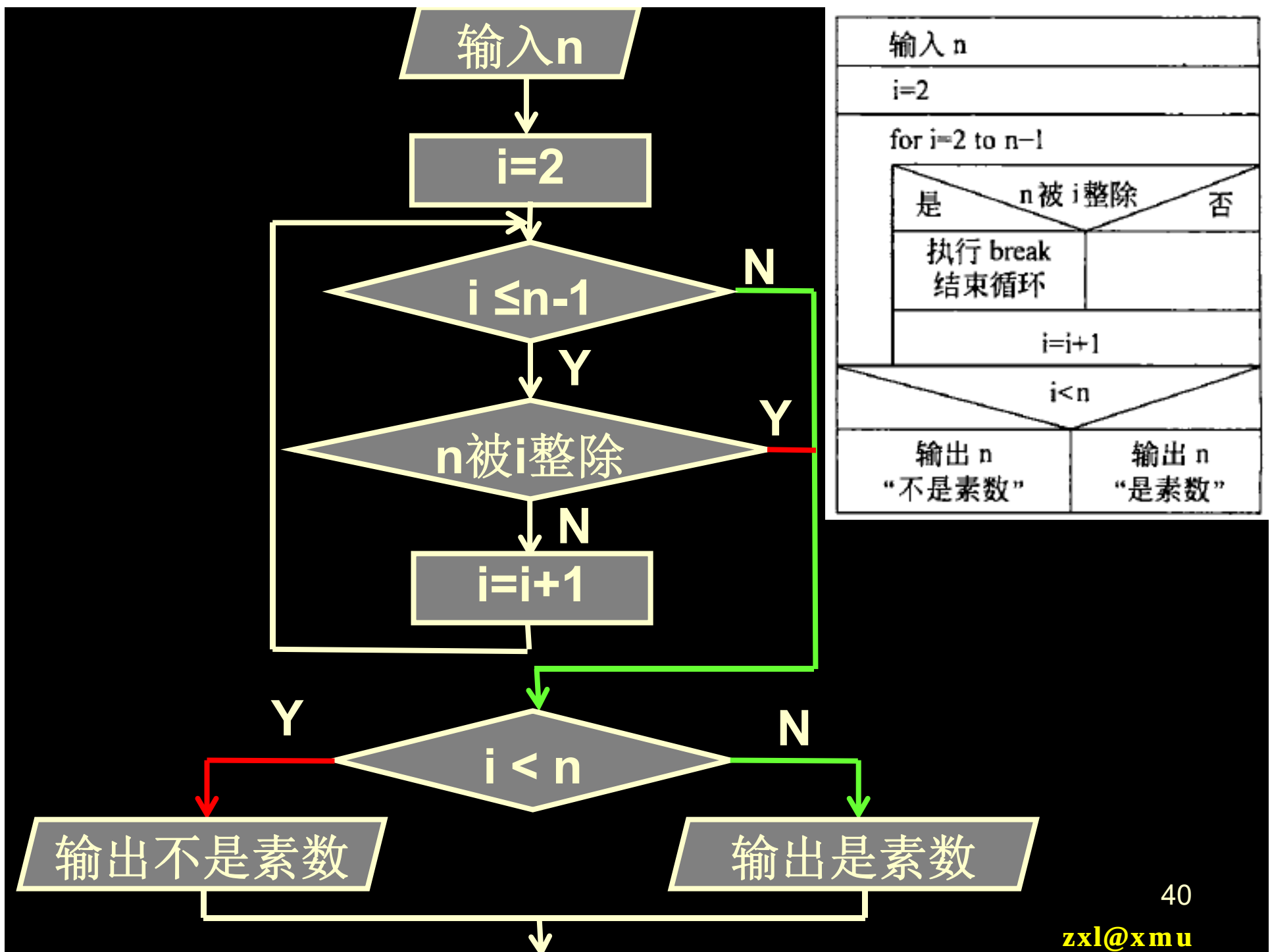
1	1	2	3
5	8	13	21
34	55	89	144
233	377	610	987
1597	2584	4181	6765
10946	17711	28657	46368
75025	121393	196418	317811
514229	832040	1346269	2178309
3524578	5702887	9227465	14930352
24157817	39088169	63245986	102334155

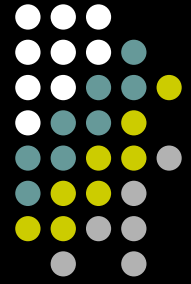
【例5.9】输入一个大于3的整数 n ，判定它是否素数(prime，又称质数)。



【解题思路】

- 让 n 被 i 整除(i 的值从2变到 $n-1$)
- 如果 n 能被 $2 \sim (n-1)$ 之中任何一个整数整除，则表示 n 肯定不是素数，不必再继续被后面的整数除，因此，可以提前结束循环
- 注意：此时 i 的值必然小于 n





```
#include <stdio.h>
```

```
int main()
```

```
{ int n,i;
```

$\sqrt{n}$

```
    printf("n=?"); scanf("%d",&n);
```

```
    for (i=2;i<=n-1;i++)
```

```
        if(n%i==0)
```

```
            break;
```

代码可改进

if (i<n)

n=?17

17 is

n=?327

327 is not

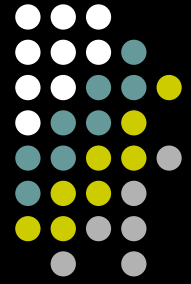
```
        printf("%d is not\n",n);
```

```
    else
```

```
        printf("%d is\n",n);
```

```
    return 0;
```

```
}
```



```
#include <stdio.h>
int main()
{ int n,i,k;
  printf("n=?"); scanf("%d",&n);
  for (i=2;i<=k;i++)
    if(n%i==0)
      break;
  if (i<=k)
    printf("%d is not\n",n);
  else
    printf("%d is\n",n);
  return 0;
}
```

$\sqrt{n}$

$k = \sqrt{n}$

【例5.10】 求100~200间的全部素数。



【解题思路】

- 使用例5.9的算法
- 在例5.9程序中只要增加一个外循环，先后对100~200间的全部整数一一进行判定即可

(详见P137)

.....

```
for(n=101,m=0;n<=200;n=n+2)
{
```

```
    k=sqrt(n);
    for (i=2; i<=k; i++)
        if (n%i==0) break;
    if (i>=k+1)
```

```
    {
        printf(" %d ",n);
        m=m+1;
```

```
    }
    if(m%10==0) printf("\n");
```

```
}
```

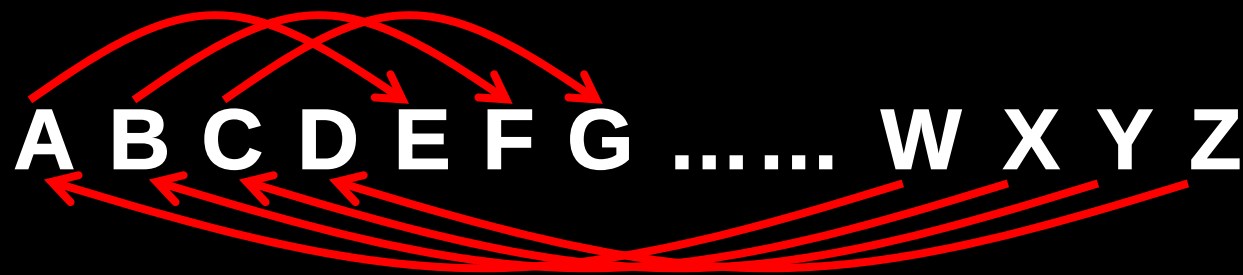
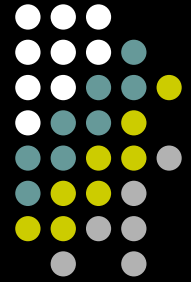
.....

只对奇数进行检查

控制每行输出10个数据



【例5.11】译密码。为使电文保密，往往按一定规律将其转换成密码，收报人再按约定的规律将其译回原文。



- 非字母字符保持原状不变
- 输入一行字符，要求输出其相应的密码

【解题思路】问题的关键有两个：



(1) 决定哪些字符不需要改变，哪些字符需要改变，如果需要改变，应改为哪个字符？

(2) 怎样使c改变为所指定的字母？ `c=getchar();`

[处理的方法] 输入一个字符给字符变量c，先判定它是否字母(包括大小写)，

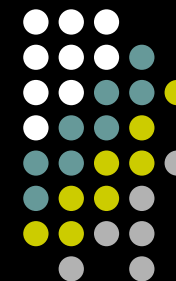
```
if ((c>='a' && c<='z') || (c>='A' && c<='Z'))
```

- 若不是字母，不改变c的值；
- 若是字母，则还要检查它是否' W' 到' Z' 的范围内
(包括大小写字母)

```
if (c>='W' && c<='Z' || c>='w' && c<='z')
```

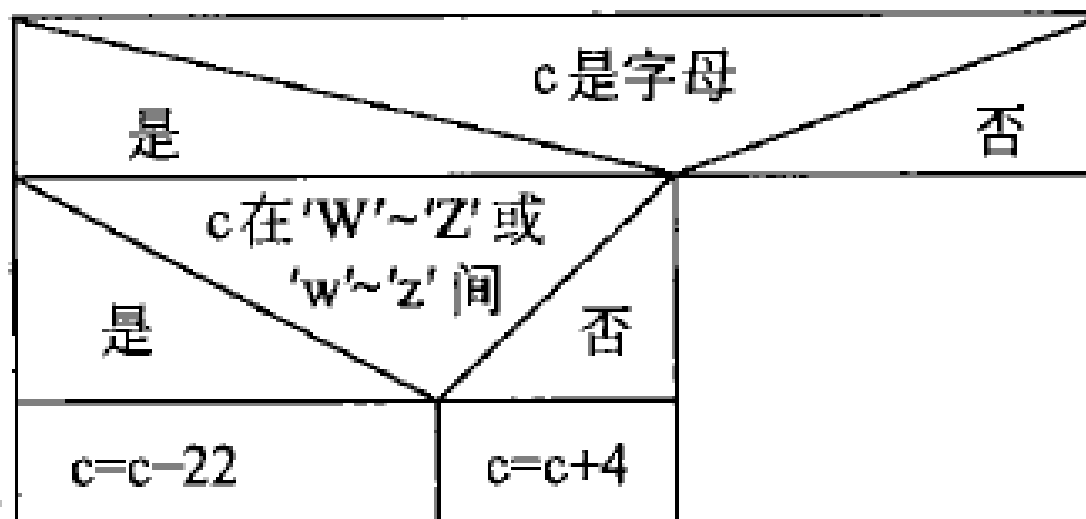
- 如不在此范围内，则变量c的值改为其后第4个字母。
- 如果在' W' 到' Z' 的范围内，则应将它 `c=c+4;`
A~D(或a~d)之一的字母。

```
c=c+4-26;
```



输入一个字符给 c

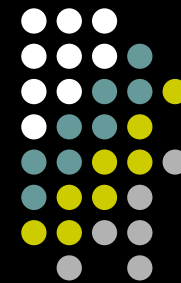
当 c 不是换行符



输出 c 的字符值

输入一个字符给 c

(详见P139)



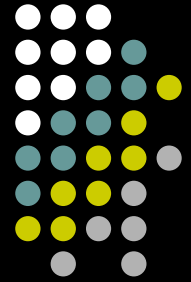
.....

```
char c;
c=getchar();
while (c!='\n')
{
    if ((c>='a' && c<='z') || (c>='A' && c<='Z'))
    {
        if (c>='W' && c<='Z' || c>='w' && c<='z')
            c=c-22;
        else
            c=c+4;
    }
    printf(" %c",c);
    c=getchar();
}
```

.....

```
China!
Glmre!
```

(详见P140)



.....

```
char c;
```

```
while ((c=getchar())!='\n')
```

```
{
```

```
    if ((c>='A' && c<='Z') || (c>='a' && c<='z'))
```

```
    {
```

```
        c=c+4;
```

```
        if (c>'Z' && c<='Z'+4 || c>'z')
```

```
            c=c-26;
```

不能少!

```
    }
```

```
    printf(" %c",c);
```

```
}
```

.....